

MISAKA PALW: A Peer-to-Peer Verifiable LLM Work System

検証者スラッシングを備えた非同期有用計算プロトコル

Watanabe Misaka

MISAKA

Draft v0.1 – August 2026

要旨. 大規模言語モデルの推論を分散ネットワークの有用計算として利用するには、出力がもっともらしいことだけでなく、指定されたモデル、重み、入力および実行規則に整合する計算証跡を、第三者が安価に検査できなければならない。さらに、計算者だけを罰し、検証者を無条件に信用する設計では、虚偽の合格、虚偽の不合格、投票コピー、共謀および未応答が新たな信頼点になる。

本稿は、MISAKA の PALW を、runtime-independent adapter、決定的 canonicalization、logits・中間 activation・GEMM trace の Merkle commitment、commit 後の予測不能 challenge、ランダムに選ばれた検証委員会、非同期 settlement、および双方の bond によって構成する。検証者は commit-reveal 形式で再計算値を提出し、単なる多数派との不一致ではなく、署名付き二重投票、未開示、証跡不提出、または決定的な dispute referee により虚偽と確定した報告に対して slash される。多数派が少数派を処罰できないよう、誤判定の slash は客観的証拠または対話型検証ゲームの最終結果に限定する。

本方式は LLM 計算を逐次ブロック生成のクリティカルパスから分離し、確定した PALW credit のみを報酬または DNS/finality weight に反映する。安全性は、challenge による不正検出確率、委員会捕捉確率、監査確率および slash 可能資本の積が、不正利益を上回るという経済条件に依存する。本稿は実装可能なメッセージ形式、状態遷移、罰則式、計算量および段階的導入手順を示すが、物理的な経過時間や消費電力そのものを暗号的に証明するものではない。

1. Introduction

インターネット上の LLM 推論は、通常、計算を実行した事業者と、その事業者が返した出力を信用することで成立する。分散型の有用計算ネットワークでは、この信頼を暗号的 commitment と経済的責任へ置き換えなければならない。計算者が最終 token だけを予測し、中間計算を省略し、別モデルを用い、過去の結果を再利用し、または安価な近似器を利用できるなら、報酬は有用計算ではなく shortcut に支払われる。

一方、計算結果を検証する validator も信用できない。validator は計算を実行せずに多数派へ追従でき、prover と共謀して虚偽の合格を提出でき、正しい prover を虚偽の不合格で恐喝でき、または challenge を受諾した後に応答しないことができる。したがって必要なのは、prover と validator の双方が署名付き証跡と bond を差し出し、誤りが客観的に確定した場合に双方が損失を負う仕組みである。

本稿では、次の構成を提案する。

1. LLM runtime と consensus-critical verification を分離し、vLLM、llama.cpp、TensorRT 系 runtime 等を共通 adapter の後ろへ置く。

- canonical logits、中間 activation、および選択された GEMM/演算 trace を chunk 化し、複数の Merkle root に commit する。
- commitment の確定後に future BlockDAG anchor と randomness beacon から challenge を導出し、検査位置の事前予測を防ぐ。
- validator を bond-weighted VRF で選び、各 validator に重複 challenge と固有 challenge を割り当て、commit-reveal で再計算値を提出させる。
- 誤判定の slash を多数決ではなく、署名付き直接証拠または対話型 dispute referee によって確定し、正直な少数派を保護する。

PALW は「指定計算を物理的に何秒実行したか」を直接証明しない。代わりに、予測不能な箇所を後から開示・再計算できる一貫した execution witness を保持していることを検査し、shortcut や虚偽報告の期待利益を bond と slash により負にする。これは、計算の所有を確率的に検査し、その検査行為自体も slashable にする方式である。

2. System Model

ネットワークは次の役割からなる。

Requester

モデル、入力、最大出力長、報酬、期限および security tier を指定して job を発行する。

Prover / Miner

指定 runtime profile で推論を実行し、出力、execution witness および proof commitment を生成する。

Verifier

challenge で指定された token、layer、chunk または演算 tile を独立に再計算し、署名付き report を提出する。

Auditor

primary committee の結果とは独立に、後段 challenge を検査し、必要に応じて dispute を開始する。

Referee

dispute を単一の決定的 micro-step まで二分探索し、chain 上で検証可能な最終判定を生成する。

BlockDAG / DNS finality layer

commitment、randomness anchor、committee assignment、settlement、slash receipt および成熟済み credit を記録する。

攻撃者は複数の prover と validator を制御し、bond を分割し、賄賂を支払い、メッセージを遅延させ、モデルの shortcut、cache、precomputation または近似実装を利用できるものとする。攻撃者は collision-resistant hash、使用する署名方式、VRF、および finality 後の randomness anchor を破れない。さらに、consensus 対象 runtime profile は同一 conformance vector に対して同一 canonical bytes を生成し、最終 referee が扱う PALW virtual machine の micro-step は決定的であると仮定する。

permissionless network では「独立した実在人物」を暗号的に判定できない。このため本方式は人格ではなく slash 可能な資本を Sybil cost とし、単一 operator が多数の鍵へ bond を分割しても総リスクが減らないよう、選出 weight cap、job ごとの bond lock、未成熟報酬の拘束および相関 fault multiplier を用いる。

3. Jobs and Work Certificates

job J は次の値を含む。

$$J = (\text{id}, M, P, x, n_{\max}, e_{\text{start}}, e_{\text{deadline}}, V_J, \tau_J), \quad (1)$$

ここで M は model manifest、 P は runtime/canonicalization profile、 x は入力 commitment、 $n_{\max}$ は最大生成 token 数、 V_J は job value、 τ_J は security tier である。model manifest は少なくとも model identifier、weights hash、tokenizer hash、quantization profile および sampling rule を含む。sampling を許可する場合、乱数 seed は job に固定し、validator が同一 token sequence を再現できなければならない。

prover は推論中に次の三種類の witness を生成する。

- token ごとの canonical logits または protocol が指定する logits subset、
 - protocol が指定する layer/chunk の canonical activation、
 - attention、normalization、GEMM 等を分割した deterministic trace chunk。
- 生の浮動小数点 tensor をそのまま hash してはならない。profile P は scale s 、bit width b 、丸め規則および

saturation を固定し、canonical integer への写像を定める。

$$Q_{s,b}(z) = \text{clip}_{[-2^{b-1}, 2^{b-1}-1]}(\text{RNE}(sz)). \quad (2)$$

profile は conformance suite を通過した backend のみを許可する。異なる backend が丸め境界で異なる bytes を生成する場合、その profile は consensus 用に有効化してはならない。許容誤差による validator ごとの局所判定は consensus rule に持ち込まず、最終的には bit-exact な canonical bytes を比較する。

witness leaf の一例を次に示す。

$$\ell_{i,\lambda,c} = H(\text{PALW-LEAF} \parallel \text{id} \parallel i \parallel \lambda \parallel c \parallel Q(z_{i,\lambda,c}) \parallel Q(y_{i,\lambda,c})), \quad (3)$$

ここで i は token index、 λ は layer/opcode、 c は chunk または tile index、 z と y は当該 micro-step の入力および出力である。prover は local witness を dispute window 終了まで保持し、chain には次の root のみを提出する。

$$R_{\text{tok}} = \text{MerkleRoot}(\ell_0^{\text{tok}}, \dots, \ell_{n-1}^{\text{tok}}), \quad (4)$$

$$R_{\text{act}} = \text{MerkleRoot}(\ell_0^{\text{act}}, \dots, \ell_{a-1}^{\text{act}}), \quad (5)$$

$$R_{\text{trace}} = \text{MerkleRoot}(\ell_0^{\text{trace}}, \dots, \ell_{t-1}^{\text{trace}}). \quad (6)$$

proof commitment は概念的に次の構造を持つ。

```

PALWProofV1 {
  algo_id
  model_id
  weights_hash
  tokenizer_hash
  runtime_profile
  quantization_profile
  canonicalization_version
  input_hash
  output_hash
  token_root
  activation_root
  trace_root
  prover_pubkey
  epoch
  nonce
  prover_signature
}

```

この commitment は、最終文章だけでなく、指定モデルの途中状態と演算 trace に prover を拘束する。もっとも、root の存在だけでは計算実行を保証しない。安全性は、commit 後に選ばれる challenge、witness の data availability、独立再計算および slash によって得られる。

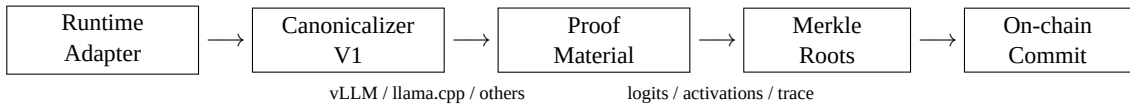


Figure 1: runtime と verification layer の分離。consensus は runtime 内部 API ではなく canonical witness format に依存する。

4. Commitment Timestamping and Challenge

challenge は proof commitment が finality anchor に取り込まれた後に決定する。job J の commitment hash を C_J 、将来 epoch の finalized BlockDAG anchor を $F_{e+\Delta}$ 、独立 randomness beacon を $B_{e+\Delta}$ とすると、challenge seed は

$$r_J = H(\text{PALW-CHALLENGE} \parallel \text{id} \parallel C_J \parallel F_{e+\Delta} \parallel B_{e+\Delta}) \quad (7)$$

で与えられる。 Δ は commitment の inclusion と beacon の予測不能性を確保する。prover が commitment を複数作って有利な challenge を選ぶ grinding を防ぐため、同一 job に対する有効 commitment 数を制限し、commitment ごとに非返還 fee または追加 bond を要求する。

challenge set χ_J は domain-separated PRF と rejection sampling により生成する。

$$\chi_J = \{(i_h, \lambda_h, c_h)\}_{h=1}^{q_J} = \text{Sample}(r_J, P, n, q_J). \quad (8)$$

q_J は固定値ではなく、job value、proof credit、prover の fault history、witness の大きさおよび security tier に応じて増加させる。低価値の市場 job と、DNS/finality weight に利用する consensus-tier work を同じ challenge 数で扱ってはならない。

validator ごとに、全員が共有する overlap challenge と、validator 固有の private challenge を割り当てる。overlap は結果比較に使われ、private challenge は他者の report をコピーする free-riding を難しくする。private assignment は validator の VRF output と r_J から導出し、vote reveal まで他 validator に予測されにくくする。

5. Network

ネットワークの標準処理は次の通りである。

1. requester は job、報酬 escrow、model manifest および期限を broadcast する。
2. prover は job を受諾し、prover bond を lock して推論と witness 生成を行う。
3. prover は output hash、三つの Merkle root、および署名を含む PROOF_COMMIT を broadcast する。
4. commitment が finality anchor に含まれた後、future randomness から challenge と verifier committee を決定する。
5. verifier は assignment を受諾すると per-job bond を lock し、指定箇所を独立に再計算する。
6. verifier は verdict と再計算値の hash を salt とともに commit し、commit window 終了後に reveal する。
7. quorum が成立した場合、job は provisional pass または provisional fail となるが、報酬と PALW credit は直ちに成熟しない。
8. dissent、fail report、mandatory audit または random audit が存在する場合、expanded audit または dispute game を開始する。
9. dispute window が終了し、未解決 challenge がなければ settlement certificate を発行し、報酬と credit を成熟させる。
10. objective evidence により prover または verifier の不正が確定した場合、slash receipt を発行し、bond、pending reward および future eligibility を更新する。

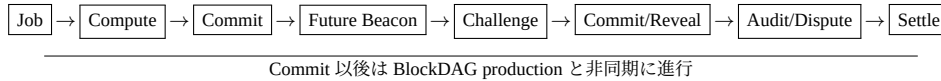


Figure 2: PALW settlement の時系列。LLM 検証は block production の逐次クリティカルパスに入れない。

新しい block は pending job の完全検証を待たない。pending credit は transfer、committee weight、DNS/finality weight または複利的選出 weight に利用できず、settlement 後にのみ mature credit へ移る。後日の slash は過去の BlockDAG を巻き戻すのではなく、未成熟報酬、bond および将来 weight に反映する。

6. Verifier Selection and Reports

validator i は global bond B_i^G を lock し、active profile ごとの conformance test を満たす。各 job の committee は、future seed に対する VRF により選ぶ。

$$(y_i, \pi_i) = \text{VRF}_{sk_i}(r_J \parallel \text{VERIFY} \parallel \text{round}), \quad (9)$$

y_i が bond-weighted threshold より小さい validator が候補となる。単一鍵への集中を防ぐため、選出 weight は $w_{\max}$ で cap し、prover key、同一 job の auditor key、未解決 slash 中の key および cooldown 中の key を除外する。VRF proof π_i は assignment eligibility とともに検証される。

committee size を k 、pass threshold を m とする。ただし、多数派は provisional outcome を決めるだけであり、少数派の slash を決めない。validator report は次を含む。

```

VerifierReportV1 {
  job_id
  verification_round
  verifier_pubkey
  assignment_hash
  verdict                // PASS or FAIL
  recomputed_values_root
  mismatch_index         // required for FAIL
  runtime_profile
  vote_commitment
  salt
  verifier_signature
}

```

commit phase では

$$C_i^V = H(\text{id} \parallel \text{round} \parallel v_i \parallel R_i^{\text{recomp}} \parallel \text{salt}_i) \quad (10)$$

のみを提出し、reveal phase で verdict、root および salt を開示する。FAIL report は少なくとも一つの具体的 mismatch index と再計算値 commitment を含まなければならず、根拠のない FAIL bit は無効である。PASS report も単なる bit ではなく、assigned challenge の canonical values をまとめた root を含む。

commit-reveal は他 validator の verdict を見て追従する行為を抑制するが、計算実行そのものを保証しない。このため private challenge、overlap challenge、mandatory audit、報酬成熟遅延および false-report slashing を組み合わせる。

7. Verifier Bonds and Slashing

validator の責任資本は、global bond、job ごとの lock および未成熟報酬からなる。

$$K_{i,J} = B_i^G + B_{i,J} + R_i^{\text{pending}}. \quad (11)$$

assignment を受諾する前であれば validator は無罰で辞退できる。受諾して $B_{i,J}$ を lock した後は、期限内の vote commit、reveal、および dispute witness 提出が義務となる。

多数派との不一致だけを slash 条件にしてはならない。捕捉された多数派が正直な少数派を処罰できるからである。slash は、次のいずれかの objective evidence が存在するときに限る。

違反種別	客観的証拠	参考 penalty class
Equivocation	同一 job、round、assignment に対する相反する二つの有効署名	bond の 100%、ejection
Commit 後の未 reveal	有効 vote commitment と reveal deadline の経過	小～中、反復で増加
Assignment timeout	assignment 受諾証明と response deadline の経過	小、liveness penalty
Malformed opening	commitment と一致しない salt、root、Merkle opening	中～大
Witness withholding	dispute 中に要求された committed witness を期限内に提出しない	大、敗訴扱い
False accept	referee が invalid と確定した challenge に PASS を署名	大～全額
False reject	referee が valid と確定した challenge に虚偽 mismatch を署名	大～全額
Profile-wide fault	多数の独立 node で同一 profile/opcode の系統的不一致	即時 slash せず quarantine

Table 1: validator slash の根拠。数値は protocol parameter であり、表の強度は相対分類である。

違反 class を c とし、job value を V_J 、当該 job で得る reward/credit value を R_J 、過去 fault score を h_i とすると、slash 額を

$$S_{i,J} = \min \{K_{i,J}, \lambda_c B_{i,J} + \alpha_c V_J + \beta_c R_J + \gamma_c h_i\} \quad (12)$$

とする。 $\lambda_c, \alpha_c, \beta_c, \gamma_c$ は severity ごとの consensus parameter である。同一 job で objective false report が n_c 件確定した場合、 관련된 共謀の外部性を反映して

$$S'_{i,J} = \min \left\{ K_{i,J}, S_{i,J} \left(1 + \eta \frac{n_c - 1}{k - 1} \right) \right\} \quad (13)$$

と増額できる。ただし software/profile の共通障害で mass slash が発生しないよう、異常率が profile fault threshold を超えた場合は profile を quarantine し、false-report slash を一時停止して deterministic conformance investigation へ移す。

slash 額は burn、成功した challenger、正直な verifier/auditor の追加報酬、および audit reserve に分割する。

$$S'_{i,J} = S_{\text{burn}} + S_{\text{challenge}} + S_{\text{honest}} + S_{\text{reserve}}. \quad (14)$$

自己 challenge による slash farming を抑えるため、 $S_{\text{challenge}}$ は実費と上限額で cap し、slash の最大部分は burn または共通 reserve に送る。liveness 違反は correctness 違反より軽くし、広域 partition で timeout が network-wide threshold を超えた場合は liveness slash を一時停止する。

prover にも同様に bond B_P を要求する。invalid witness、異なる model/weights、Merkle opening 不一致、output withholding または dispute witness 不提出が確定した場合、prover reward と pending credit を没収し、job value と損害に応じて B_P を slash する。prover と validator の penalty accounting は同一 settlement engine で処理するが、証拠 class は分離する。

8. Dispute Resolution

validator の虚偽を slash するには、「別の validator が違うと言った」以上の根拠が必要である。dispute は三段階で処理する。

8.1 Direct Evidence

Equivocation、commitment/opening 不一致、期限経過、および署名不正は chain 上で直接判定できる。これらには再推論を必要とせず、evidence transaction が受理されると即座に slash receipt を発行する。

8.2 Expanded Audit

primary committee に dissent がある場合、または security tier が mandatory audit を要求する場合、元 committee と重ならない expanded audit committee を future randomness から選ぶ。expanded audit は追加 challenge を再計算し、provisional outcome を更新する。expanded majority は settlement のために有用だが、それだけで dissenting validator を slash しない。

8.3 Interactive Referee

false accept または false reject の slash を求める party は appeal bond B_A を lock し、dispute を開始する。prover、challenger および対象 validator は、問題の token/layer/chunk に至る state sequence

$$s_0, s_1, \dots, s_T \quad (15)$$

の Merkle root に拘束される。両者が異なる最初の区間を binary search し、 $O(\log T)$ round で単一 transition

$$s_t \xrightarrow{\text{opcode}, w} s_{t+1} \quad (16)$$

まで縮小する。最終 transition は、固定整数 arithmetic、固定 overflow rule、固定 tensor layout および weights Merkle proof を持つ PALW virtual machine で chain 上検証する。

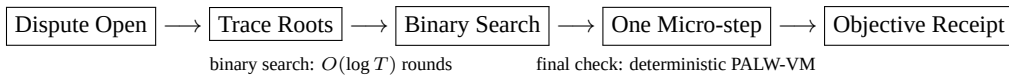


Figure 3: 誤判定を客観化する対話型 dispute。多数決ではなく最終 micro-step が slash 根拠になる。

大きな integer GEMM tile を直接再計算するコストが高い場合、有限体上の Freivalds check を補助的に用いることができる。 $C = AB$ を検査するとき、post-dispute randomness から r を選び、

$$A(Br) = Cr \quad (17)$$

を確認する。独立反復により誤受理確率を指数的に減らせる。ただし、probabilistic referee を使用する profile は error bound を明示し、最終 slash の根拠に使う場合は security parameter を chain consensus が固定する。小さな tile では exact integer execution を優先する。

witness を期限内に提出しない party は当該争点で敗訴する。referee が prover の witness を invalid と確定した場合、prover は slash され、false PASS を提出した validator も各 report に応じて slash される。referee が prover を valid と確定した場合、虚偽 mismatch を提出した validator と敗訴した frivolous appellant の appeal bond が slash される。正直な少数派は多数派に負けても、最終 receipt により保護される。

9. Incentive

job fee と protocol issuance を合わせた settlement budget を F_J とし、

$$F_J = R_P + R_V + R_A + R_D \quad (18)$$

に分ける。 R_P は prover、 R_V は primary verifier、 R_A は auditor、 R_D は dispute reserve である。validator reward は多数派と同じ bit を出したことではなく、期限内に assignment を受諾し、固有 challenge の canonical values root を提出し、必要な witness を保持し、最終 receipt と矛盾しなかったことに対して支払う。

報酬は provisional outcome ではなく settlement 後に成熟する。dissenting validator が dispute で正しいことを示した場合、その validator は通常 reward に加えて challenge bounty を得る。これにより、誤った多数派への追従よりも、独立再計算と正しい dissent の期待収益を高くする。

validator i が虚偽 report によって得る最大利益を $G_{V,i}$ 、検出・裁定確率を $p_{V,i}$ 、slash 額を $S_{V,i}$ 、失う将来 reward と cooldown value を $L_{V,i}$ とすると、正直性の経済条件は

$$p_{V,i} (S_{V,i} + L_{V,i}) > G_{V,i} \quad (19)$$

である。同様に prover について

$$p_P (S_P + L_P) > G_P \quad (20)$$

を要求する。slash を単なる固定 fee にすると、高価値 job で賄賂利益が penalty を上回る。このため式 (12) は job value、credit value および反復 fault を考慮する。

全 primary validator が共謀して同じ虚偽 PASS を出す場合、primary 内の dissent は存在しない。この場合の $p_{V,i}$ は mandatory/random audit に依存する。DNS/finality weight に利用する consensus-tier work では独立 secondary audit を必須とし、市場用途では job value に応じて audit probability a_J を設定する。低い a_J を採用するなら、その分だけ必要 bond は大きくなる。無料の安全性という便利な魔法は存在しない。

10. Reclaiming Witness Storage

full witness は巨大であるため、永続的に全 node が保存する必要はない。prover と assigned validator は dispute window D_J の間、必要な witness、Merkle path、runtime manifest および input/output data availability pointer を保持する。settlement 後、通常 node は次のみを残せばよい。

1. job と proof commitment、
2. verifier assignment と VRF proof、
3. vote commitment/reveal、
4. final settlement certificate、
5. dispute transcript の最終 root と slash receipt。

Merkle root より下の witness は pruning できる。高価値 job、profile activation test、および研究用 benchmark は archive node が長期間保持できる。input または model weight が validator へ提供されない job は public PALW verification の対象外とし、privacy-preserving execution は別 protocol version として扱う。

11. Simplified Verification

light client は LLM を実行せず、次を確認する。

1. proof commitment が finalized BlockDAG anchor に含まれること、
2. challenge seed と committee VRF proof が正しいこと、
3. vote commit/reveal と署名が一致すること、
4. quorum、audit requirement および dispute window が満たされたこと、
5. settlement certificate または objective referee receipt が有効であること。

これにより light client は、network が定めた検証手順と経済条件が実行されたことを確認できる。ただし、全 committee と auditor が捕捉され、audit が発生せず、challenge が不正箇所を外した場合まで独立に検出できるわけではない。light verification の安全性は、committee selection、bond distribution、challenge coverage、data availability および referee の健全性に依存する。

12. Calculations

12.1 Prover Fault Detection

不正 witness が全 challenge 候補の割合 f に影響し、独立に q 箇所を抽出すると、少なくとも一つを検出する確率は

$$P_{\text{detect}} = 1 - (1 - f)^q. \quad (21)$$

不正割合 f	$q = 16$	$q = 64$	$q = 256$	$q = 512$
0.5%	7.71%	27.44%	72.29%	92.32%
1.0%	14.85%	47.44%	92.37%	99.42%
5.0%	55.99%	96.25%	99.9998%	> 99.999999%
10.0%	81.47%	99.88%	> 99.999999%	$\approx 100\%$

Table 2: 単純な独立 sampling 仮定における challenge coverage。局所的 shortcut を想定すると 1 token 固定検証は不十分である。

実際の LLM trace では fault が token、layer、tile 間で相関する。したがって sampling は token だけでなく layer と opcode domain を層化し、同一種類の leaf へ偏らないようにする。 q は job ごとの最悪 fault model と benchmark に基づき決定しなければならない。

12.2 Committee Capture

adversary が eligible verifier weight の割合 ρ を持ち、 k 人中 m 人以上を捕捉すると provisional outcome を支配できるとする。独立近似では

$$P_{\geq m}(\rho) = \sum_{i=m}^k \binom{k}{i} \rho^i (1 - \rho)^{k-i}. \quad (22)$$

k	m	$\rho = 10\%$	$\rho = 20\%$	$\rho = 25\%$	$\rho = 33.3\%$
5	4	0.0460%	0.6720%	1.5625%	4.5267%
7	5	0.0177%	0.4672%	1.2878%	4.5267%
9	6	0.0064%	0.3066%	0.9995%	4.2422%
11	8	0.000125%	0.0235%	0.1188%	0.8823%

Table 3: 単純 binomial model における committee threshold 捕捉確率。実運用では bond concentration と相関 operator により独立性が弱くなる。

primary committee の捕捉確率を P_C 、audit committee の捕捉確率を P_A 、audit probability を a_J 、referee error bound を ϵ_R とする。不正割合 f の proof が受理される確率の粗い上限は

$$P_{\text{bad}} \leq (1 - f)^q + [1 - (1 - f)^q] P_C [(1 - a_J) + a_J P_A] + \epsilon_R. \quad (23)$$

第一項は challenge miss、第二項は challenge が当たっても captured committee が隠蔽し audit が救済しない場合、第三項は referee 自体の error である。consensus-tier work では $a_J = 1$ とし、primary と audit の双方を独立 seed と異なる eligible set から選ぶことが望ましい。

12.3 Validator Deterrence

validator が計算を省略して得る節約、賄賂および collusion profit の合計を $G_{V,i}$ とする。unanimous false vote の検出確率が概ね

$$p_{V,i} \geq a_J(1 - P_A) \quad (24)$$

であるなら、式 (19) から必要 slashable capital は

$$S_{V,i} + L_{V,i} > \frac{G_{V,i}}{a_J(1 - P_A)}. \quad (25)$$

a_J を 2% に落とせば、他条件が同じでも必要 penalty は利益の約 50 倍へ増える。したがって audit rate、bond、job value cap および成熟遅延は独立パラメータではない。protocol は job を受理する前に、この不等式を満たすだけの locked capital があるか確認する。

13. BlockDAG and DNS Integration

PALW verification は 10 BPS の BlockDAG production と並行に走らせる。block validity は、未成熟 PALW job の最終結果に直接依存させない。代わりに、job は次の状態を遷移する。

$$\begin{aligned} &\text{CREATED} \rightarrow \text{COMMITTED} \rightarrow \text{CHALLENGED} \rightarrow \text{VERIFYING} \\ &\rightarrow \text{PROVISIONAL} \rightarrow \text{AUDITING/DISPUTED} \rightarrow \text{SETTLED}. \end{aligned} \quad (26)$$

DNS/finality weight、validator selection reputation または protocol issuance に利用できるのは SETTLED-PASS の mature credit だけである。PROVISIONAL-PASS を利用できるようにすると、後から slash される credit で自身を validator に選び、さらに自己検証する正の feedback loop が生じる。

導入は次の段階で行う。

Stage 0 – Shadow

commitment、challenge、report、dispute を実行するが、報酬・weight へ影響させない。

Stage 1 – Rewards only

settlement 済み work に限定して報酬を与え、fork-choice と DNS weight には使わない。

Stage 2 – Capped weight

mature PALW credit を上限付きで DNS/finality weight に反映し、hash PoW floor と既存安全弁を残す。

Stage 3 – Security integration

多 backend conformance、adversarial benchmark、長期 slash simulation および emergency rollback test を通過した profile のみ、より強く security へ統合する。

protocol upgrade は algo_id、runtime_profile および canonicalization_version によって明示する。version を跨ぐ commitment、challenge または referee trace を混在させてはならない。

14. Security Considerations and Limitations

本方式の主要な安全上の注意は次の通りである。

1. 計算時間の直接証明ではない。正しい witness を知っていることを検査する。precomputation、cache、distillation または未知の shortcut が proof material を安く生成できるなら、security premise は弱くなる。
2. Canonicalization が consensus-critical である。GPU、driver、kernel、quantization および compiler 差で bytes が分岐する profile は有効化してはならない。
3. Data availability が必要である。input、output prefix、model weights および challenged witness が期限内に取得できなければ、独立再計算は成立しない。
4. Validator market の集中は committee 式を弱める。表 3 は独立 sampling の近似であり、同一 operator、同一 cloud、同一 client bug の相関を含まない。
5. Slash は objective receipt に限定する。多数決だけで誤判定を罰すると、正直な少数派が最も危険になる。

6. **Common-mode bug** と共謀を区別しにくい。profile-wide fault detector、quarantine、段階的導入および capped integration が必要である。
7. **経済安全性は value aware でなければならない**。job value が locked capital を超える場合、protocol は job を拒否するか、committee、audit rate、challenge 数および bond を増やす。

本稿は protocol design と threat accounting を示すものであり、全 opcode の deterministic VM、formal semantics、network synchrony bound、具体的な BlockDAG randomness construction および game-theoretic equilibrium の完全証明は今後の仕様書と実装監査に委ねる。

15. Conclusion

我々は、LLM の有用計算を trust ではなく、commitment、予測不能 challenge、独立再計算および双方の slashable bond によって扱う PALW protocol を提案した。prover は canonical logits、activation および execution trace の root に拘束され、validator は固有 challenge の再計算値を commit-reveal で提出する。検証結果は BlockDAG production と非同期に settlement され、成熟済み credit だけが報酬または DNS/finality weight に利用される。

本設計の中心は、validator を善意の監査者として扱わない点にある。虚偽合格、虚偽不合格、二重投票、未開示および witness withholding は slash 対象となる。ただし誤判定の罰は多数派投票では決めず、署名付き直接証拠または deterministic dispute referee によって客観化する。これにより、prover と validator の双方について

$$P_{\text{detect}} \times \text{slashable loss} > \text{cheating gain} \quad (27)$$

を満たす経済環境を構成できる。

安全な実装には、bit-exact profile、trace data availability、value-aware bond、committee independence、mandatory consensus-tier audit、profile quarantine および長期 adversarial benchmark が必要である。これらを満たさない状態で PALW を直ちに fork-choice の絶対条件へ組み込むべきではない。まず shadow と reward-only で測定し、その後に上限付き weight へ進めるのが妥当である。

A. Consensus Message Summary

```
JOB_CREATE
  job_id, model_manifest, runtime_profile, input_hash,
  max_tokens, value, security_tier, deadlines, requester_sig

PROOF_COMMIT
  job_id, output_hash, token_root, activation_root, trace_root,
  prover_bond, epoch, nonce, prover_sig

VERIFY_ASSIGN
  job_id, round, verifier_pubkey, vrf_proof,
  overlap_challenges, private_challenges, response_deadline

VOTE_COMMIT
  job_id, round, assignment_hash, vote_commitment, verifier_sig

VOTE_REVEAL
  job_id, round, verdict, recomputed_values_root,
  mismatch_index, salt, verifier_sig

DISPUTE_OPEN
  job_id, disputed_report, claim_type, appeal_bond,
  trace_root, challenger_sig

DISPUTE_BISECT
  job_id, round, interval, midpoint_state_hash,
  merkle_opening, party_sig

REFEREE_RESULT
  job_id, disputed_step, opcode, input_openings,
  output_opening, result, referee_receipt

SETTLE
```

```
job_id, final_result, mature_credit, reward_distribution,
released_bonds, settlement_certificate
```

SLASH

```
offender_pubkey, job_id, evidence_class, objective_receipt,
slash_amount, distribution, cooldown_until
```

B. Reference Testnet Parameters

次の値は security guarantee ではなく、実験開始点である。benchmark によって更新されなければならない。

Parameter	Initial range	Purpose
Primary committee (k, m)	(7, 5) または (9, 6)	provisional outcome
Consensus-tier audit	100% secondary audit	unanimous collusion の外部検査
Market-tier audit a_J	5–25%, value dependent	bond と連動して設定
Base challenge q	64–256	token/layer/tile stratified sampling
Dispute window D_J	1–24 hours equivalent	witness availability と finality の分離
Vote commit/reveal	separate short windows	copy voting と bandwagon の抑制
Verifier job lock	max loss に比例	$G/[p(1 - P_A)]$ 条件を満たす
Equivocation slash	100% + ejection	objective double-sign evidence
Timeout slash	1–10% progressive	liveness、partition circuit breaker あり
False verdict slash	50–100% dynamic	referee receipt 後のみ
Credit maturity	dispute 終了後	self-reinforcing verification の防止

Table 4: Testnet 用の参考範囲。production parameter は実測と formal analysis を要する。

References

- [1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. <https://bitcoin.org/bitcoin.pdf>
- [2] Ambient, “Ambient Litepaper V1,” Proof of Logits and asynchronous validation design. https://ambient.xyz/Ambient_Litepaper_V1.pdf
- [3] Ambient Documentation, “On-Chain: VerificationState,” verifier assignments, token ranges, Merkle root, verified tokens and output hash. <https://docs.ambient.xyz/On-Chain-27ee653486a380c1b652d649e90d6636>
- [4] S. Goldberg, L. Reyzin, D. Papadopoulos, and J. Včelák, “Verifiable Random Functions (VRFs),” RFC 9381, 2023. <https://www.rfc-editor.org/rfc/rfc9381>
- [5] J. Teutsch and C. Reitwießner, “A Scalable Verification Solution for Blockchains,” 2019. <https://arxiv.org/abs/1908.04756>
- [6] R. Freivalds, “Fast Probabilistic Algorithms,” Mathematical Foundations of Computer Science 1979, LNCS 74, pp. 57–69, 1979. https://doi.org/10.1007/3-540-09526-8_5
- [7] R. C. Merkle, “Protocols for Public Key Cryptosystems,” 1980 IEEE Symposium on Security and Privacy, pp. 122–134, 1980. <https://doi.org/10.1109/SP.1980.10006>
- [8] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” FIPS 204, 2024. <https://doi.org/10.6028/NIST.FIPS.204>